

Programming for Business (CL-2016)

LAB INSTRUCTOR
OWAIS HAMEED

Student Assessment and Marks Distribution:

Assessments	Percentage
• Sessional (Lab Task, Quiz, Project)	20%
• Midterm Examination	30%
• Final Exam	50%

Week 1 Objectives:

- Introduction to Python Programming for Business
- Importance of programming in business decision-making
- Examples from finance, marketing, and operations
- Overview of Python language
- Installation Anaconda and setup Jupyter Notebook
- Writing and running the first program

Introduction of the Class



Introduction to Python Programming for Business

- What is Programming?
 - Programming is giving instructions to a computer to solve problems or perform tasks.
- Business Analogy:
 - Accountant tells clerk step-by-step how to prepare a report.
 - Manager gives SOPs to employees.
 - Programmer gives instructions to a computer.
- Computers are fast but dumb, they only do what we tell them.

Why Python for FinTech?

- Python is a high-level, easy to learn programming language.
- Widely used in:
 - Finance & Banking
 - Data Analysis
 - FinTech startups
 - AI & Machine Learning
 - Automation

Key Features:

- Easy English like syntax
- Free & open-source
- Huge community support
- Works on Windows, Mac, Linux
- Created by Guido van Rossum
 - He is a Dutch programmer who began working on Python in the late 1980s and first released it in February 1991.

Traditional Tools Vs. Python

- Excel → Limited automation
- SPSS(IBM) → Expensive
- Manual work → Time-consuming
- **Python Advantages:**
 - Automates reports
 - Analyzes large financial data
 - Predicts trends
 - Reduces human error
 - Saves time & cost

Python Applications in FinTech

Real-World Use Cases:

- Banking → Fraud detection, Credit scoring.
- Stock Market → Price prediction, Portfolio optimization.
- Finance → Profit/Loss analysis, Budget forecasting.
- E-Commerce → Sales analysis, Customer behavior.
- AI & ML → Chat bots, Risk assessment.
- Companies using Python like Google, Netflix, PayPal & Bloomberg.

Python vs. Other Languages

Language	Difficulty	Use
Python	Easy	Business, AI, Finance
Java	Hard	Large systems
C++	Very Hard	Games, OS
R	Medium	Statistics

Importance of programming in business decision-making

- In today's digital economy, business decisions are no longer based on intuition alone. Organizations rely on data, algorithms, and automation to make accurate, fast, and scalable decisions.
- Programming is the backbone that enables this transformation.
- For FinTech professionals, programming is not optional, it is a core business skill.
- Data-driven decisions outperform intuition driven decisions. Programming turns data into decisions.

What Is Programming in a Business Context?

Programming in business means:

- Writing instructions for computers to process data
- Automating repetitive and complex tasks
- Building models, dashboards, and decision systems
- Enabling real-time analysis and predictions

Traditional Decision Making vs. Programming Based Decisions

Traditional Approach

- Manual calculations
- Limited data
- Slow decisions
- Error-prone
- Experience-based

Programming-Based Approach

Automated computations

Big data & real-time data

Fast & scalable decisions

More accurate & consistent

Data + logic-based

Importance of Programming Specifically in FinTech

- Risk Management
 - Credit scoring models
 - Fraud detection algorithms
- Algorithmic Trading
 - Automated buy/sell decisions
 - High-frequency trading
- Digital Payments & Banking
 - Transaction processing
 - Fraud prevention

Examples from Finance, Marketing, and Operations

Finance Decision-Making Example

❑ **Credit Risk Assessment:** A bank must decide whether to approve or reject a loan.

- **Data Used:** Income, credit score, transaction history, defaults
- **Programming Role:**
 - Python analyzes historical loan data
 - Machine learning predicts default probability
- **Decision Outcome:**
 - Approve, reject, or send for manual review

Impact: Reduced bad loans and faster approvals.

Marketing Decision-Making Examples

❑ Customer Segmentation

- **Data Used:** Age, income, spending behavior, app usage
- **Programming Role:**
 - Python clusters customers into segments
- **Decision Outcome:**
 - Different offers for different customer groups

Impact: Higher customer engagement and sales.

Operations Decision-Making Examples

❑ Inventory Management

- **Data Used:** Sales history, demand patterns, lead time
- **Programming Role:**
 - Forecast demand using time-series models
 - Automatically generate reorder alerts
- **Decision Outcome:**
 - Optimal inventory levels

Impact: Reduced stock outs.

Key Takeaway from Examples

Across finance, marketing, and operations.

Programming:

- Converts raw data into insights
- Reduces uncertainty in decisions
- Enables automation and scalability
- Better code leads to better business decisions.

Overview of Python language

Basic Components of Python Language

Data Types: Integer (int), Floating point (float), String (str), Boolean (bool).

Operators

- Arithmetic (+, -, *, /, %, **)
- Comparison (>, <, ==, >=, <=)
- Logical (and, or, not)

Data Types

Variable Types	Basic Examples	Real World Examples
Integer	Z = 1	Release_Year= 2003
Float	Z = 1.23	Rating = 8.2
String	Z = "Python"	Movie = "Finding Nemo"
Boolean	Z = True	Is_Released = True
List	Z = ["Alice", "Lily"]	Actors = ["Elen DeGeneres", "Alexander Gould"]
Dictionary	Z = {"Name": "Alan", "ID": 1}	Box_Office_Collection= {"US": "339.7M", "UK": "67.1M"}
Tuple	Z= ("Monday", 26, "June")	Release_Date = ("30th", "May", 2003)
Set	Z= {2,3,5}	Genre = {"Animation", "Family", "Adventure"}

Control Structures in Python

Conditional Statements

- if
- if-else
- if-elif-else

Used for decision-making

Loops

- for loop
- while loop

Used for repetition and automation.

Functions in Python

Functions help:

- Organize code
- Reuse logic
- Improve readability

In business:

- Calculating profit
- Computing interest
- Generating reports

Python Libraries (Power of Python)

Python becomes powerful because of its libraries:

Core Libraries

- **NumPy** – Numerical computations
- **Pandas** – Data analysis and manipulation
- **Matplotlib** – Data visualization
- **Scikit-learn** – Machine learning

What Is a Variable?

- A variable is a name given to a memory location that stores a value.
- In business and FinTech, variables store:
 - Prices
 - Profits
 - Interest rates
 - Customer balances
 - Risk values
- Think of a variable as a **container** that holds data.

Variable Assignment in Python

In Python, variables are assigned using the **assignment operator (=)**.

- **Syntax:**

- `variable_name = value`

- **Example:**

- `profit = 50000`

- `interest_rate = 0.12`

- `bank_name = "HBL"`

- Python automatically determines the **data type** of the value.

Rules for Naming Variables

- Python variable names must follow these rules:
 - Can contain letters, numbers, and underscores (_)
 - Must start with a letter or underscore
 - Are case-sensitive (Profit $\neq$ profit)
 - Cannot start with a number
 - Cannot contain spaces
 - Cannot use Python keywords
 - Valid Examples: total_sales, _balance
 - Invalid Examples: 2profit ✗ interest rate ✗ class ✗

1. Must start with a letter or underscore

✓ first_name ✗ 2nd_name ✓ _third_name

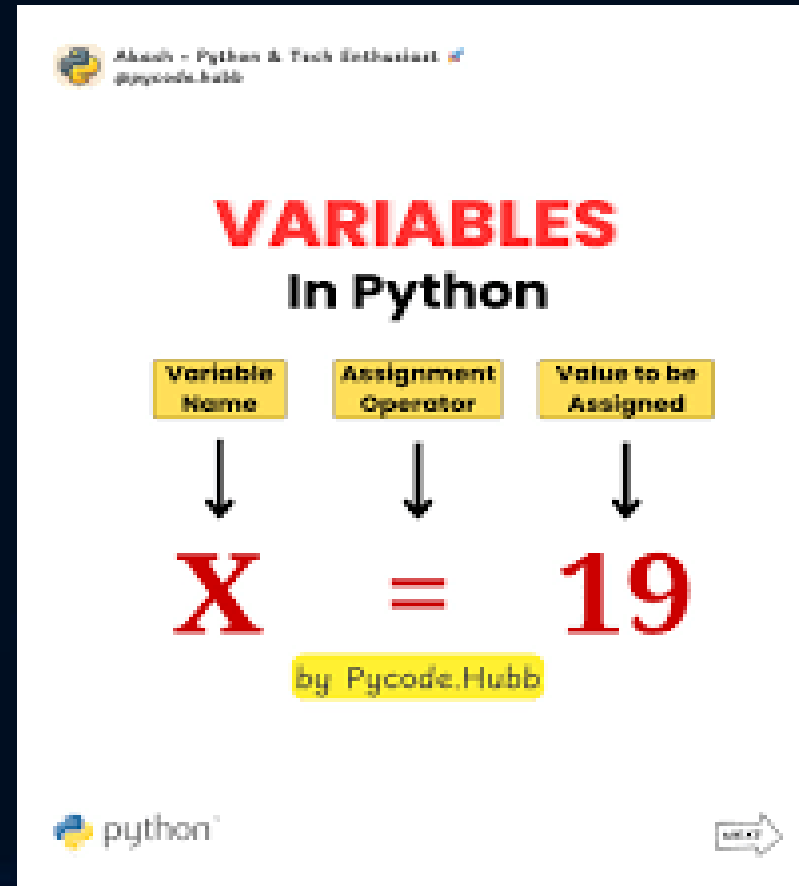
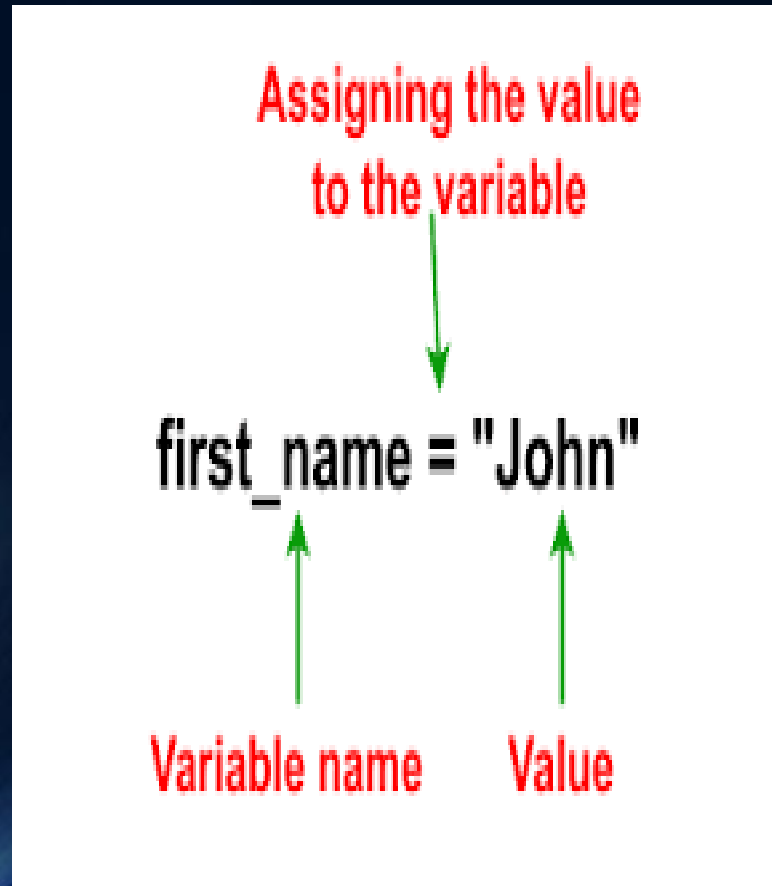
2. Must only contain letters, numbers, and underscores

✓ my_var123 ✗ my~wavy~var ✗ var*var

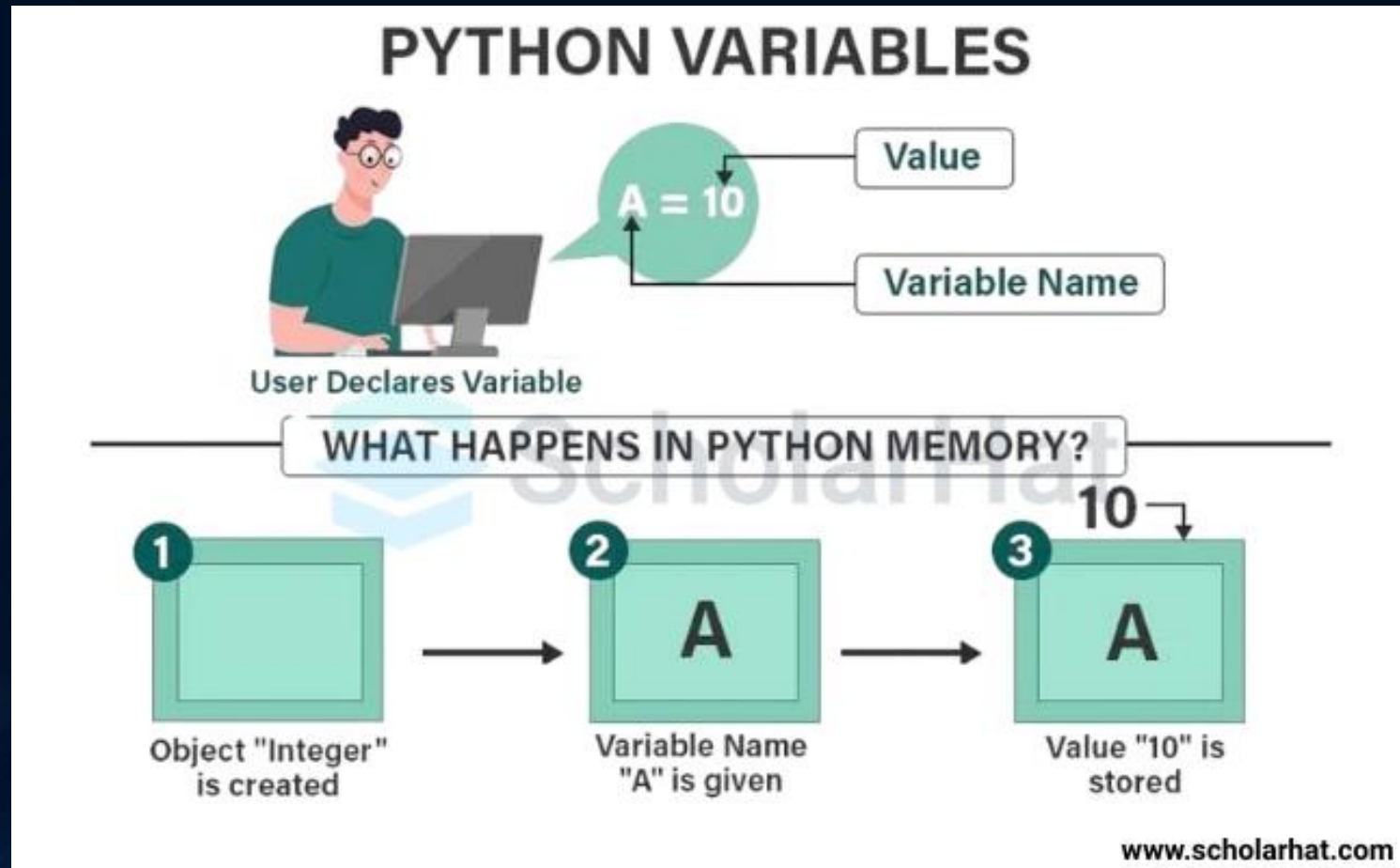
3. Case-sensitive

myvariable $\neq$ myVariable $\neq$ MYVARIABLE

Creation of Variable:



How Python works?



Assigning Different Data Types

Integer (int)

- `customers = 120`

Floating Point (float)

- `roi = 7.5`

String (str)

- `company = "FinTech Ltd"`

Boolean (bool)

- `loan_approved = True`

Multiple Variable Assignment

- Python allows assigning multiple variables in one line.
- Example: `revenue, expenses, profit = 500000, 350000, 150000`
- This is useful in financial calculations and data processing.

Assigning Same Value to Multiple Variables

- `q1 = q2 = q3 = 0`
- Used when initializing values before calculations.

Reassigning Variables

- Variables can be updated anytime.
- Example: `balance = 500`, `balance = balance + 2000`
- Used in deposits, profits, and incremental calculations.

Variable Assignment with User Input

- `age = int(input("Enter your age: "))`
- Useful in: Customer forms, Financial calculators, Interactive applications.

Common Mistakes by Beginners

- ✗ Using spaces in variable names
- ✗ Confusing = with ==
- ✗ Using unclear names (x, y) instead of meaningful ones
- ✓ Always use descriptive variable names in business code.

Best Practices for Business & FinTech Use

- meaningful names (interest_rate, not ir)
- Follow snake_case naming convention
- Keep variable names short but clear

Installation and setup Jupyter Notebook



Writing and running the first program

- Example 1: Hello World

```
print("Hello World!")
```

```
print("Welcome to FinTech with Python")
```

- Example 2: Business Calculation

```
revenue = 500000
```

```
cost = 350000
```

```
profit = revenue – cost
```

```
print("Profit is:", profit)
```

- Example 3: Simple Interest

```
principal = 100000
```

```
rate = 10.5
```

```
time = 2
```

```
interest = (principal * rate * time) / 100
```

```
print("Simple Interest:", interest)
```

- Type Checking Data Types:

```
print(type(time))
```

```
print(type(rate))
```

```
print(type("FinTech"))
```

Input & Output

```
name = input("Enter company name:")  
print(name)  
print(len(name))
```

- Take sales and cost from user and calculate profit.

```
sales = int(input("Enter total sales: "))  
cost = int(input("Enter total cost: "))  
profit = sales - cost  
print("Profit:", profit)
```

Monthly Salary Calculation

- Objective: Real-life business problem
- Problem: Calculate monthly salary.

```
basic_salary = int(input("Enter basic salary: "))
```

```
allowance = int(input("Enter allowance: "))
```

```
total_salary = basic_salary + allowance
```

```
print("Total Monthly Salary:", total_salary)
```

Task 1:

- Addition (Total Amount)
- Problem: Calculate total amount received from two income sources.

```
income1 = 50000
```

```
income2 = 30000
```

```
total_income = income1 + income2
```

```
print("Total Income:", total_income)
```

Task 2:

- Subtraction (Profit / Loss)
- Problem: Calculate profit.

```
revenue = 120000
```

```
expenses = 85000
```

```
profit = revenue – expenses
```

```
print("Profit:", profit)
```

Task 3:

- Multiplication (Sales Growth)
- Problem: Calculate total sales for multiple units.

```
unit_price = 1500
```

```
units_sold = 40
```

```
total_sales = unit_price * units_sold
```

```
print("Total Sales:", total_sales)
```

Task 4:

- Division (Average Revenue)
- Problem: Calculate average monthly revenue.

```
annual_revenue = 600000
```

```
months = 12
```

```
average_monthly = annual_revenue / months
```

```
print("Average Monthly Revenue:", average_monthly)
```

Task 5:

- Floor Division (Installments)

Problem: Calculate equal installments.

```
total_amount = 100000
```

```
installments = 6
```

```
per_installment = total_amount // installments
```

```
print("Per Installment Amount:", per_installment)
```

- **Modulus (Remaining Amount)**

Problem: Calculate remaining amount after installments.

```
remaining_amount = total_amount % installments
```

```
print("Remaining Amount:", remaining_amount)
```

Task 6

- Exponent Operator (Compound Growth – Basic)
- Problem: Calculate investment growth.

```
principal = 100000
```

```
growth_factor = 1.10
```

```
years = 2
```

```
future_value = principal * (growth_factor ** years)
```

```
print("Future Value of Investment:", future_value)
```

```
print(f"Future Value of Investment: {future_value:.3f}")
```

Lab 1 Final Task:

1. Display company information.
2. Take revenue and expenses from the user.
3. Calculate profit or loss.
4. Calculate total sales cost.
5. Calculate average monthly revenue.
6. Calculate simple interest.
7. Calculate number of full installments and remaining balance.
8. Calculate future value using power operator upto 5 decimal places.